UNIT I – Introduction to AI & Problem-Solving Agents

AI & ML for Mechanical Engineering (23A03701)

1. What is Artificial Intelligence?

➤ Definition

AI = the science and engineering of making machines (computers, robots, software) do things that would need intelligence if done by humans – learning, reasoning, perceiving, planning, understanding language.

The term was coined by **John McCarthy** in **1956** (Dartmouth Conference).

➤ Four views of AI (Russell & Norvig)

	Human-like	Rational (ideal)
Thinking	Thinking humanly – cognitive modelling	Thinking rationally – "laws of thought", logic
Acting	Acting humanly – **Turing Test**	Acting rationally – **rational agent** (most used)

Turing Test (1950): a human interrogator asks written questions; if he cannot tell whether the answers come from a human or a machine, the machine is said to be intelligent.

➤ Why AI for Mechanical Engineers?

- Predictive maintenance of pumps, turbines, bearings
- Robotics & autonomous vehicles, path planning
- Design optimisation, generative design, CFD / FEA surrogates
- Smart manufacturing (Industry 4.0), quality inspection by vision
- Process control – fuzzy / adaptive controllers

2. Problems of AI (areas / task domains)

Category	Tasks
Mundane tasks	Perception (vision, speech), natural language understanding & generation, commonsense reasoning, robot control
Formal tasks	Games (chess, checkers), mathematics (geometry, logic, theorem proving), integral calculus
Expert tasks	Engineering design, scientific analysis, medical diagnosis, financial analysis, fault finding

- Humans do *mundane* tasks easily but computers find them hard; *formal* tasks look hard but are easier to program.
- Typical AI problems need **search**, **knowledge** and **abstraction**.

➤ Key difficulties

- Huge search spaces (combinatorial explosion)
- Knowledge is vast, incomplete, uncertain and changing
- Real world is noisy – sensors give partial information
- Real-time response needed (robots, vehicles)

3. The AI Technique

An **AI technique** is a method that exploits knowledge in a way that makes the problem easier to solve.

➤ Properties of knowledge

- It is **voluminous** (huge)
- It is **hard to characterise accurately**
- It is **constantly changing**
- It differs from data – it is **organised** so it can be **generalised** (captures regularities) and **understood** by the people who provide it
- It can be **easily modified** to correct errors and reflect changes

- It can be used in many situations even if incomplete – helps ***narrow*** the possibilities

Three important AI techniques:

- ***Search*** – provides a way of solving problems for which no direct approach is available (try alternatives systematically).
- ***Use of knowledge*** – solve complex problems by exploiting the structure of objects involved.
- ***Abstraction*** – separate important features from unimportant details so that the process is not overwhelmed.

➤ Underlying assumption (Physical Symbol System Hypothesis – Newell & Simon)

"A physical symbol system has the necessary and sufficient means for general intelligent action."

4. Tic-Tac-Toe Problem – three approaches

Aim: show how a ***better data structure + more knowledge*** make a program smarter. Board = 3×3, players X and O, 3 in a line wins.

➤ Program 1 – Lookup table (simple)

- Board = vector of 9 elements (0 = blank, 1 = X, 2 = O). Treat as a ***ternary number***.
- Convert to decimal → index into a table of 3^9 = 19683 entries; each entry stores the ***next board (move)***.

Drawbacks: huge memory, someone must fill all entries by hand, not extendable (3-D tic-tac-toe needs 3^{27} entries), not intelligent.

➤ Program 2 – Magic-square / turn-based

- Board = 9-element vector, but 2 = blank, 3 = X, 5 = O. Turn counter 1…9.
- Functions: ***Make2*** (play centre if free, else a non-corner), ***Posswin(p)*** (returns the square that wins for p, else 0 – product of a line = 18 for X, 50 for O), ***Go(n)*** (play in square n).
- Strategy by turn: odd turns X plays, even turns O. Win if possible, else block, else take centre, else corner.

Advantages: less space, strategy can be explained. Disadvantage: it is ***not flexible*** – cannot be generalised to other games.

➤ Program 3 – Game tree / minimax style

- Board position = 9-element vector plus a ***list of positions reachable*** by one legal move, plus a ***rating*** (how likely to lead to a win).
- Look ahead: generate all successors, evaluate each, back up values; choose the move with best rating.

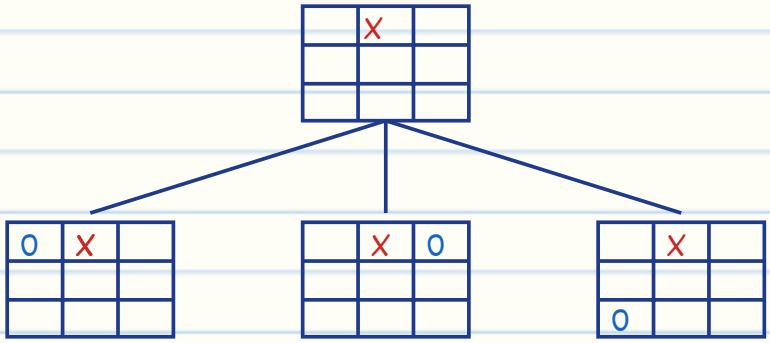


Fig: a partial game tree for Tic-Tac-Toe (X to move, O replies)

Program 3 is ***slower*** but more ***general*** and lets us use ***look-ahead*** – the same idea is used in chess programs. It shows that more knowledge + better structure → better program.

5. Intelligent Agents

➤ Agent

An ***agent*** is anything that perceives its environment through sensors and acts upon it through actuators.

- ***Human agent:*** eyes, ears (sensors); hands, legs (actuators)
- ***Robotic agent:*** cameras, IR, encoders (sensors); motors, grippers (actuators)
- ***Software agent:*** keystrokes, files, packets (sensors); screen display, files (actuators)

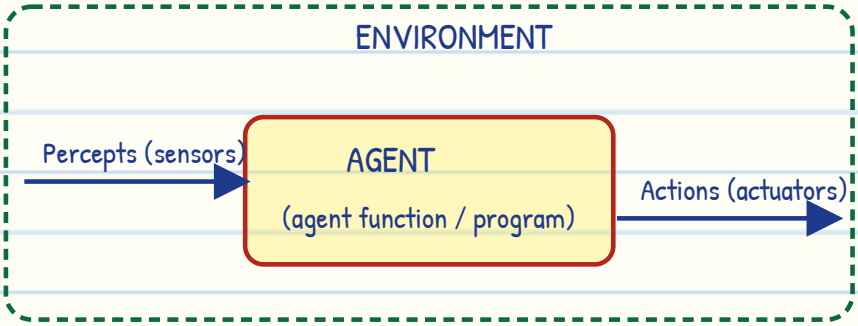


Fig: Agent interacting with environment

- ***Percept*** = agent's perceptual input at one instant; ***percept sequence*** = complete history of percepts.
- ***Agent function*** maps percept sequence → action. ***Agent program*** = concrete implementation running on the ***architecture*** (hardware). Agent = architecture + program.

➤ Rational agent

For each possible percept sequence, a rational agent selects the action expected to ***maximise its performance measure***, given the evidence from percepts and built-in knowledge.

Rationality depends on: (1) performance measure, (2) prior knowledge of environment, (3) actions it can perform, (4) percept sequence to date.

> ***Example – Vacuum-cleaner agent:*** world = squares A, B; percepts = [location, Clean/Dirty]; actions = Left, Right, Suck. Rule: if Dirty → Suck; else move to the other square. Performance measure: +1 per clean square per time-step.

Omniscience ≠ rationality. A rational agent maximises *expected* performance; it also needs ***information gathering***, ***learning*** and ***autonomy***.

6. Nature of Environments – PEAS

To design an agent we first specify the ***task environment*** as PEAS: Performance measure, Environment, Actuators, Sensors.

Agent	P	E	A	S
Automated car / taxi	Safe, fast, legal, comfortable, profit	Roads, traffic, pedestrians	Steering, accelerator,	Cameras, LIDAR, GPS, speedometer

			brake, horn	
Industrial robot (pick & place)	% parts correctly placed, cycle time	Conveyor belt, parts	Jointed arm, gripper	Camera, joint-angle sensors, force sensor
Medical / fault diagnosis system	Correct diagnosis, minimal cost	Patient / machine, staff	Display of tests, diagnosis	Keyboard entry of symptoms, vibration data

➤ Properties of task environments

Property	Meaning (easy ⟷ hard)
Fully ⟷ Partially observable	Sensors give complete state at each time? (chess – full; taxi – partial)
Deterministic ⟷ Stochastic	Next state fully determined by current state + action? (chess – det.; taxi – stochastic). *Strategic* = deterministic except other agents' actions.
Episodic ⟷ Sequential	Are experiences divided into independent episodes? (part inspection – episodic; chess – sequential)
Static ⟷ Dynamic	Does environment change while agent deliberates? (crossword – static; taxi – dynamic). *Semi-dynamic*: only the score changes with time (timed chess).
Discrete ⟷ Continuous	Finite number of states/actions? (chess – discrete; robot arm – continuous)
Single ⟷ Multi-agent	One agent or several (competitive / cooperative)?
Known ⟷ Unknown	Does agent know the "laws" of the environment?

Hardest case: partially observable, stochastic, sequential, dynamic, continuous, multi-agent (e.g. driving).

7. Structure of Agents (types)

➤ (a) Simple reflex agent

- Acts only on the ***current percept*** using ***condition–action rules*** (if … then …). No memory.
- Example: thermostat, "if car in front is braking → brake", automatic door.
- Works only if environment is ***fully observable***; may loop forever in partial observability.

```
function SIMPLE-REFLEX-AGENT(percept)
  state  ← INTERPRET-INPUT(percept)
  rule   ← RULE-MATCH(state, rules)
  return rule.ACTION
```

➤ (b) Model-based reflex agent

- Keeps an ***internal state*** (model) of the part of the world it cannot see – "how the world evolves" + "what my actions do".
- Example: car remembers that the vehicle behind exists although it is not in the camera now.

➤ (c) Goal-based agent

- Has an explicit ***goal*** (desirable situation). Chooses actions that achieve the goal using ***search and planning*** – "what will happen if I do A?"
- More ***flexible***: changing the goal changes behaviour without rewriting rules. Example: route-finding robot, taxi going to a destination.

➤ (d) Utility-based agent

- Goals give only a yes/no (happy / unhappy). ***Utility function*** maps a state to a real number = degree of happiness.
- Chooses the action with ***maximum expected utility***; handles conflicting goals (speed vs safety) and uncertainty. Example: choose the route that is fast *and* safe *and* cheap.

➤ (e) Learning agent

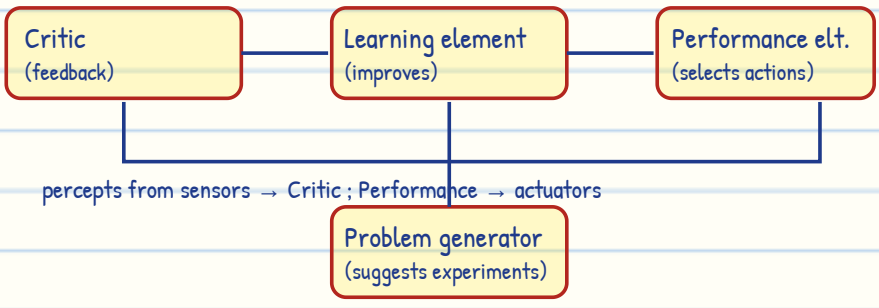


Fig: General learning agent

- ***Performance element*** – selects external actions (the whole agent of earlier types).
- ***Learning element*** – makes improvements using knowledge from the critic.
- ***Critic*** – tells how well the agent is doing vs a fixed performance standard.

- **Problem generator** – suggests exploratory actions leading to new experiences.

➤ Quick comparison

Agent	Uses	Memory?	Looks ahead?
Simple reflex	Condition-action rules	No	No
Model-based	Internal state + rules	Yes	No
Goal-based	Goal + search	Yes	Yes
Utility-based	Utility function	Yes	Yes (best action)
Learning	Feedback + critic	Yes	Improves with experience

8. Defining the Problem as State Space Search

➤ State space

The set of **all possible states** reachable from the start state by applying legal **operators (rules)**. A problem is solved by finding a path from the initial state to a goal state.

➤ Steps to define a problem

1. **Define the state space** – initial state, possible states.
2. **Specify initial state(s).**
3. **Specify goal state(s)** (one or many).
4. **Specify the rules / operators** – legal actions, with their preconditions and effects; keep them general, not too detailed.

A well-defined problem has: **initial state, actions (successor function), goal test, path cost**. A solution with the lowest path cost is **optimal**.

> **Example 1 – Water-jug problem:** 4-litre jug (x) and 3-litre jug (y), no markings, pump available. Goal: exactly 2 litres in the 4-litre jug.
> State = (x, y), start (0,0), goal (2, n). Rules include: fill 4-jug (x,y) → (4,y); fill 3-jug → (x,3); empty either; pour 3-jug into 4-jug until full or empty: (x,y) → (x+y,0) if x+y≤4; and so on.
> **Solution path:** (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0) ✔

Example 2 – 8-puzzle: 3×3 board, 8 tiles + blank. State = tile positions; operators = move blank Up/Down/Left/Right; goal = ordered arrangement; path cost = 1 per move.

Example 3 – Missionaries & Cannibals: 3 + 3 on a bank, boat takes 2. Cannibals must never outnumber missionaries on either bank. State = (M, C, boat side).

Example 4 – Travelling Salesman / robot path / assembly sequence – same idea: states = partial tours/placements.

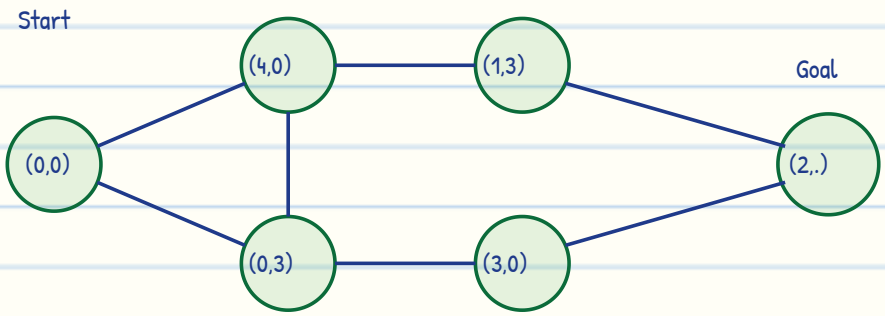


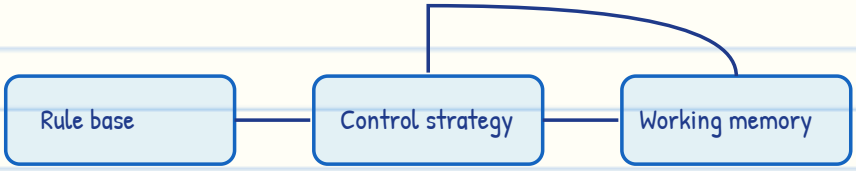
Fig: part of the water-jug state space (nodes = states, edges = operators)

9. Production Systems

➤ Definition

A ***production system*** consists of:

1. ***A set of production rules*** – each is ***condition → action*** (left side = pattern; right side = operation).
2. ***One or more knowledge bases / databases*** – current facts (working memory) and global state.
3. ***A control strategy*** – decides which rule to apply when many match; resolves conflicts (and avoids looping).
4. ***A rule applier*** – matches the pattern and executes the action.



match → select → apply → repeat until goal

➤ Control strategy requirements

- ***Causes motion*** – it must not stand still (must apply some rule each cycle).
- ***Systematic*** – do not repeat the same state needlessly (avoid infinite loops).

➤ Classes of production systems

- ***Monotonic*** – applying a rule never prevents a later rule that was applicable earlier from being applied.
- ***Non-monotonic*** – not monotonic.
- ***Partially commutative*** – if a sequence of rules transforms state x to y, any *permutation* of the allowable rules also transforms x to y.
- ***Commutative*** – both monotonic and partially commutative.

	Monotonic	***Non-monotonic***
Partially commutative	Theorem proving	Robot navigation
Not partially commutative	Chemical synthesis	Bridge / chess

Commutative systems are the easiest – ***ignorable*** (e.g. theorem proving) problems can be solved without backtracking; ***recoverable*** (8-puzzle) need backtracking; ***irrecoverable*** (chess) need planning/look-ahead.

10. Problem Characteristics

Before choosing a method, analyse the problem with these questions:

1. ***Is the problem decomposable?*** Can it be broken into independent sub-problems? (symbolic integration – yes; blocks world – interacting sub-goals).
2. ***Can solution steps be ignored or undone?*** *Ignorable* (theorem proving), *recoverable* (8-puzzle), *irrecoverable* (chess) – decides if backtracking/planning is needed.

3. ***Is the universe predictable?*** Certain-outcome (8-puzzle) vs uncertain-outcome (bridge, robot with noisy sensors).
4. ***Is a good solution absolute or relative?*** Any-path problem (find one proof) vs best-path problem (TSP – shortest route) – best-path searches all paths.
5. ***Is the solution a state or a path?*** Interpretation of a sentence → just a state; water-jug → the path of operations.
6. ***What is the role of knowledge?*** Chess needs little knowledge (rules) but newspaper-story understanding needs vast knowledge.
7. ***Does the task need interaction with a person?*** Solitary (give problem, get answer) vs conversational (user feedback during solving).

11. Issues in the Design of Search Programs

- ***Direction of search – forward*** (from start to goal) or ***backward*** (goal to start); choose by branching factor, whether goals/starts are more numerous, and whether justification is needed.
- ***Choosing the right rules*** – avoid too many or too specific rules; use ***indexing*** / hashing to match quickly.
- ***Representation of nodes – tree vs graph***: graphs detect repeated states (need check of OPEN/CLOSED lists), trees may re-explore the same state.
- ***Search-tree vs search-graph*** – avoiding duplicate nodes saves effort but costs checking time.
- ***Heuristic knowledge*** – use of an ***evaluation function*** to choose which node to expand next.
- ***Size of the search space*** – combinatorial explosion; ***pruning*** is essential.
- ***Matching*** – how to pick applicable rules efficiently.
- ***Termination & completeness / optimality*** – will the search always find a solution, the best one, within time and memory?

➤ Evaluating a search strategy

- ***Completeness*** – guaranteed to find a solution if one exists?
- ***Optimality*** – finds the least-cost solution?

- ***Time complexity*** – nodes generated (b = branching factor, d = depth of goal).
- ***Space complexity*** – maximum nodes stored in memory.

12. Important Exam Questions

1. Define AI. Explain the different approaches / the Turing Test.
2. What is an AI technique? List its properties.
3. Explain the tic-tac-toe problem with the three programs and compare.
4. Define agent and rational agent. Explain with PEAS (give an example).
5. Explain the types of agent programs with block diagrams.
6. Explain the properties of task environments.
7. Formulate (a) water-jug, (b) 8-puzzle as state-space search problems.
8. What is a production system? Explain its components and categories.
9. Discuss the characteristics of a problem.
10. Discuss the issues in the design of search programs.

✎ *All the best! Revise definitions, diagrams and examples.*